BUILD AND DEPLOY OF MACHINE LEARNING APPLICATIONS
EXPERIMENT -6

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Suppress TensorFlow warnings for cleaner output
tf.keras.utils.disable_interactive_logging()


# ============================================================
# PART 1: BUILDING A SIMPLE FEEDFORWARD NEURAL NETWORK (FNN)
# ============================================================

print("--- Part 1: Building a Simple Feedforward Neural Network ---")

# 1. Load and Preprocess Dataset (Using Fashion MNIST for FNN)
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) =
fashion_mnist.load_data()

print(f"\nOriginal FNN training data shape: {x_train_fnn.shape}")
print(f"Original FNN test data shape: {x_test_fnn.shape}")

# Flatten images to 1D array
x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)

# Normalize pixel values
x_train_fnn_norm = x_train_fnn_flat / 255.0
x_test_fnn_norm = x_test_fnn_flat / 255.0

print(
    f"Flattened & Normalized FNN training data shape: "
    f"{x_train_fnn_norm.shape}"
)
print(
    f"Flattened & Normalized FNN test data shape: "
    f"{x_test_fnn_norm.shape}"
)

# 2. Build FNN Model
model_fnn = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
```

```
])

# 3. Compile Model
model_fnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("\n--- FNN Model Summary ---")
model_fnn.summary()

# 4. Train Model
print("\n--- Training FNN Model ---")

history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)

# 5. Evaluate Model
print("\n--- Evaluating FNN Model ---")

loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)

print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")

# Classification report & confusion matrix
y_pred_fnn = np.argmax(
    model_fnn.predict(x_test_fnn_norm),
    axis=-1
)

print("\n--- FNN Classification Report ---")
print(classification_report(y_test_fnn, y_pred_fnn))

print("\n--- FNN Confusion Matrix ---")

cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)

plt.figure(figsize=(10, 8))
sns.heatmap(
```

```
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)

plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

# Plot Accuracy & Loss
plt.figure(figsize=(12, 5))

# Accuracy plot
plt.subplot(1, 2, 1)

plt.plot(
    history_fnn.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history_fnn.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title('FNN Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss plot
plt.subplot(1, 2, 2)

plt.plot(
    history_fnn.history['loss'],
    label='Training Loss'
)

plt.plot(
    history_fnn.history['val_loss'],
    label='Validation Loss'
)

plt.title('FNN Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()


# ============================================================
# PART 2: CONVOLUTIONAL NEURAL NETWORK (CNN)
# ============================================================

print("\n--- Part 2: Implementing a CNN ---")

# 1. Load MNIST for CNN
(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) =
mnist.load_data()

print(f"\nOriginal CNN training data shape: {x_train_cnn.shape}")
print(f"Original CNN test data shape: {x_test_cnn.shape}")

# Reshape for channel dimension
x_train_cnn = x_train_cnn.reshape(
    x_train_cnn.shape[0],
    28,
    28,
    1
)

x_test_cnn = x_test_cnn.reshape(
    x_test_cnn.shape[0],
    28,
    28,
    1
)

# Normalize
x_train_cnn = x_train_cnn.astype('float32') / 255.0
x_test_cnn = x_test_cnn.astype('float32') / 255.0

print(
    f"Reshaped & Normalized CNN training data shape: "
    f"{x_train_cnn.shape}"
)

print(
    f"Reshaped & Normalized CNN test data shape: "
    f"{x_test_cnn.shape}"
)

num_classes_cnn = 10
```

```python
# 2. Build CNN Model
model_cnn = keras.Sequential([
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
        input_shape=(28, 28, 1)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation='relu'
    ),

    layers.Dropout(0.5),

    layers.Dense(
        num_classes_cnn,
        activation='softmax'
    )
])

# 3. Compile Model
model_cnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("\n--- CNN Model Summary ---")
model_cnn.summary()

# 4. Train Model
print("\n--- Training CNN Model ---")

history_cnn = model_cnn.fit(
    x_train_cnn,
```

```
    y_train_cnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)

# 5. Evaluate Model
print("\n--- Evaluating CNN Model ---")

loss_cnn, accuracy_cnn = model_cnn.evaluate(
    x_test_cnn,
    y_test_cnn,
    verbose=0
)

print(f"CNN Test Loss: {loss_cnn:.4f}")
print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")

# Classification report & confusion matrix
y_pred_cnn = np.argmax(
    model_cnn.predict(x_test_cnn),
    axis=-1
)

print("\n--- CNN Classification Report ---")
print(classification_report(y_test_cnn, y_pred_cnn))

print("\n--- CNN Confusion Matrix ---")

cm_cnn = confusion_matrix(
    y_test_cnn,
    y_pred_cnn
)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm_cnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)

plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

```python
# Plot Accuracy & Loss
plt.figure(figsize=(12, 5))

# Accuracy plot
plt.subplot(1, 2, 1)

plt.plot(
    history_cnn.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history_cnn.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.title('CNN Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss plot
plt.subplot(1, 2, 2)

plt.plot(
    history_cnn.history['loss'],
    label='Training Loss'
)

plt.plot(
    history_cnn.history['val_loss'],
    label='Validation Loss'
)

plt.title('CNN Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()


# ============================================================
# OPTIONAL: VISUALIZE CNN PREDICTIONS
# ============================================================

print("\n--- Sample CNN Predictions ---")
```

```
class_names_mnist = [str(i) for i in range(10)]

plt.figure(figsize=(10, 10))

for i in range(25):

    plt.subplot(5, 5, i + 1)

    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    plt.imshow(
        x_test_cnn[i].reshape(28, 28),
        cmap=plt.cm.binary
    )

    true_label = y_test_cnn[i]
    predicted_label = y_pred_cnn[i]

    color = (
        'green'
        if true_label == predicted_label
        else 'red'
    )

    plt.xlabel(
        f"True: {class_names_mnist[true_label]}\n"
        f"Pred: {class_names_mnist[predicted_label]}",
        color=color
    )

plt.suptitle(
    "Sample CNN Predictions "
    "(Green: Correct, Red: Incorrect)",
    y=1.02,
    fontsize=16
)

plt.tight_layout(
    rect=[0, 0, 1, 0.98]
)

plt.show()
```

```
--- Part 1: Building a Simple Feedforward Neural Network ---

Original FNN training data shape: (60000, 28, 28)
Original FNN test data shape: (10000, 28, 28)
```

```
Flattened & Normalized FNN training data shape: (60000, 784)
Flattened & Normalized FNN test data shape: (10000, 784)

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/
dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)


--- FNN Model Summary ---



--- Training FNN Model ---

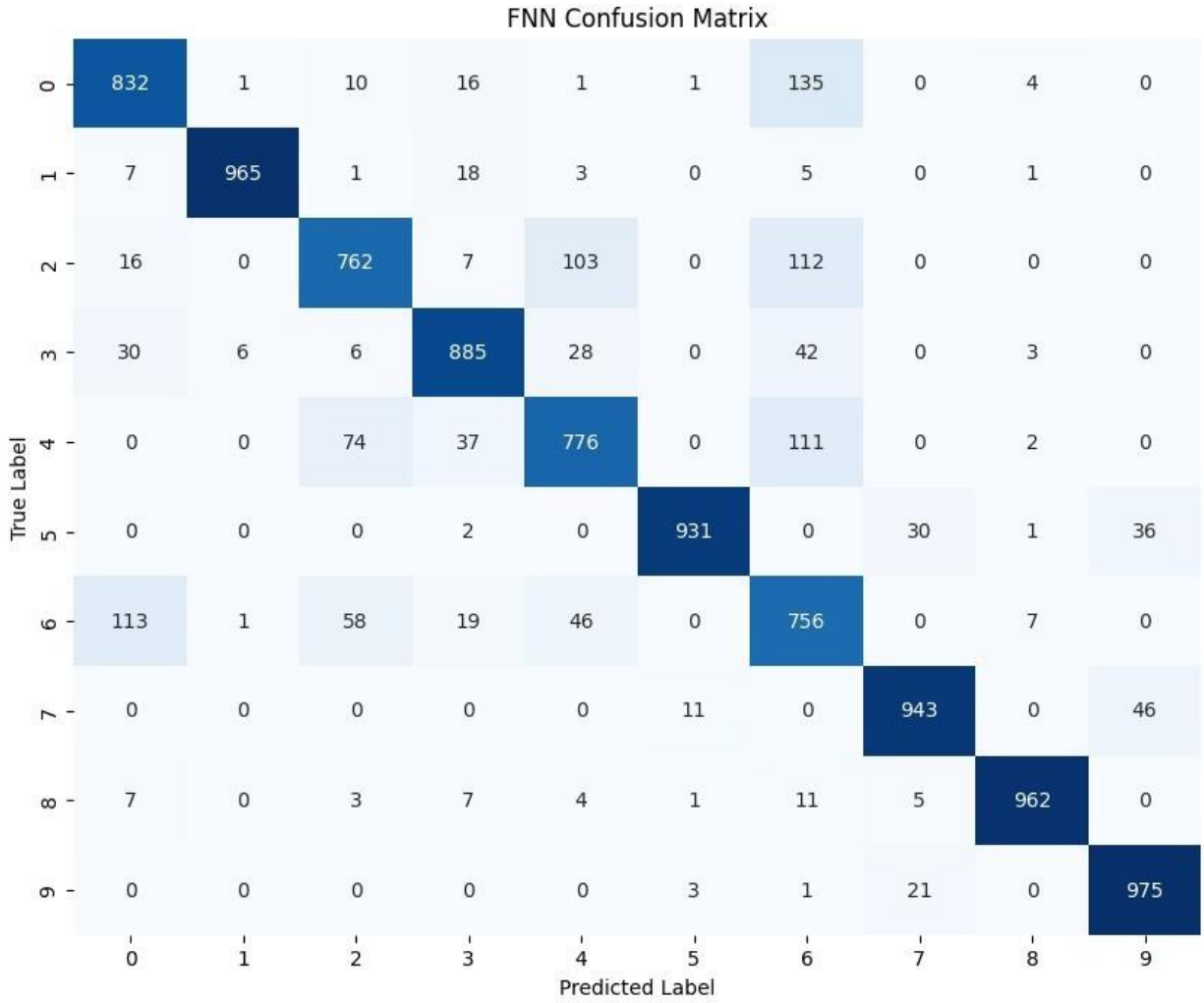
--- Evaluating FNN Model ---
FNN Test Loss: 0.3418
FNN Test Accuracy: 0.8787

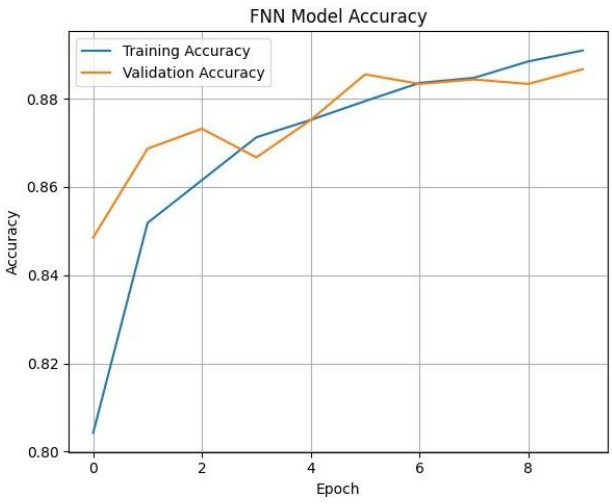
--- FNN Classification Report ---
              precision    recall  f1-score   support

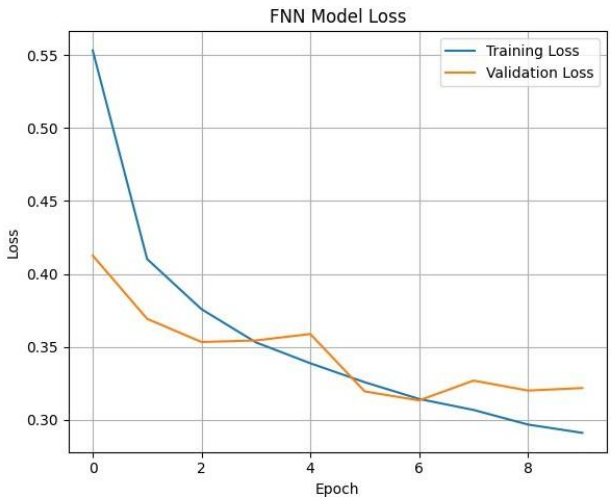
           0       0.83      0.83      0.83      1000
           1       0.99      0.96      0.98      1000
           2       0.83      0.76      0.80      1000
           3       0.89      0.89      0.89      1000
           4       0.81      0.78      0.79      1000
           5       0.98      0.93      0.96      1000
           6       0.64      0.76      0.70      1000
           7       0.94      0.94      0.94      1000
           8       0.98      0.96      0.97      1000
           9       0.92      0.97      0.95      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000


--- FNN Confusion Matrix ---
```








```
--- Part 2: Implementing a CNN ---
```

```
Original CNN training data shape: (60000, 28, 28)
Original CNN test data shape: (10000, 28, 28)
Reshaped & Normalized CNN training data shape: (60000, 28, 28, 1)
Reshaped & Normalized CNN test data shape: (10000, 28, 28, 1)

--- CNN Model Summary ---

/usr/local/lib/python3.13/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)



--- Training CNN Model ---

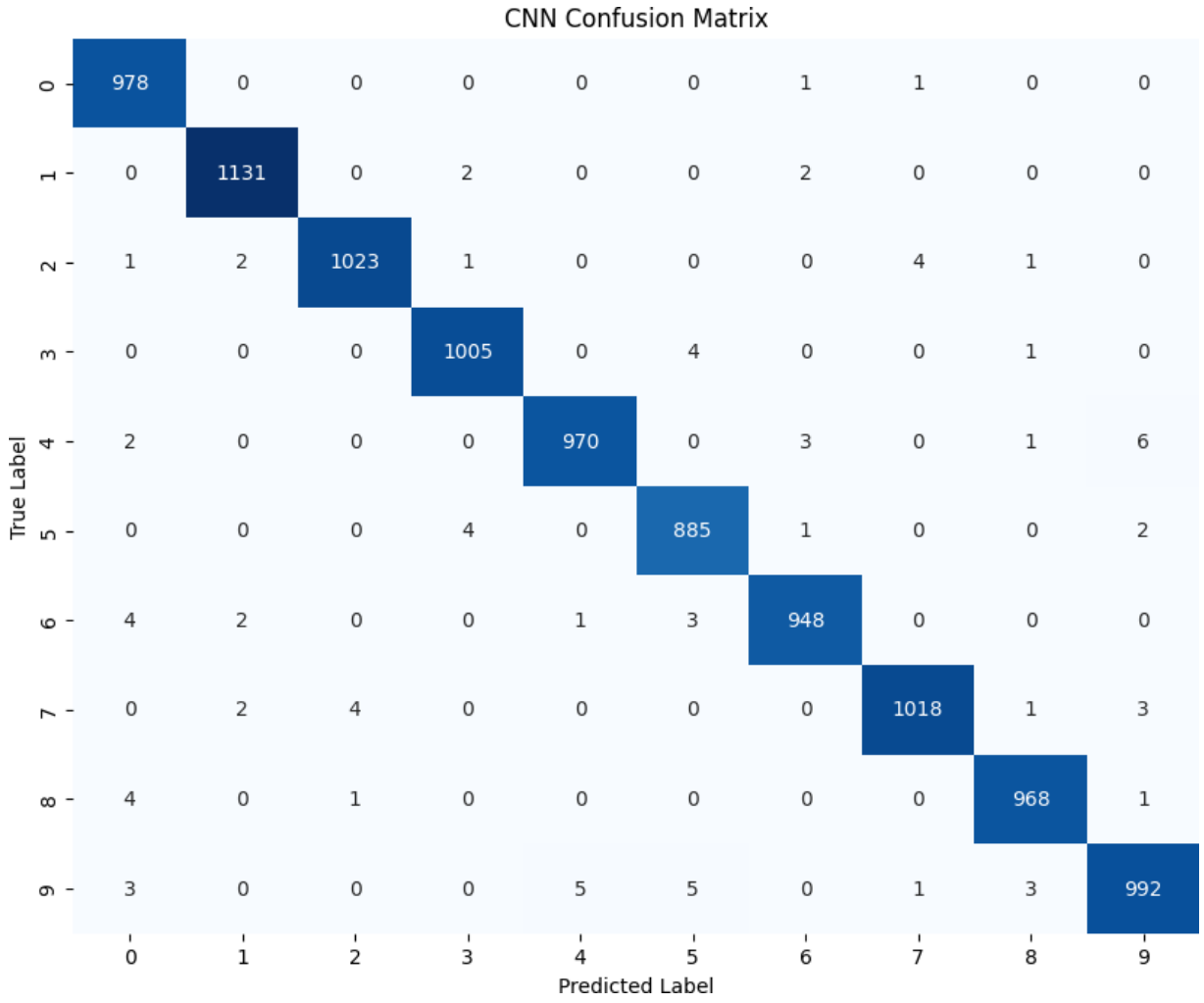
--- Evaluating CNN Model ---
CNN Test Loss: 0.0234
CNN Test Accuracy: 0.9918

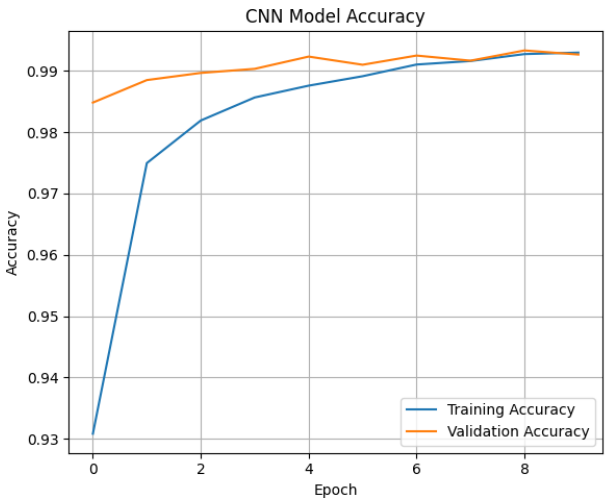
--- CNN Classification Report ---
              precision    recall  f1-score   support

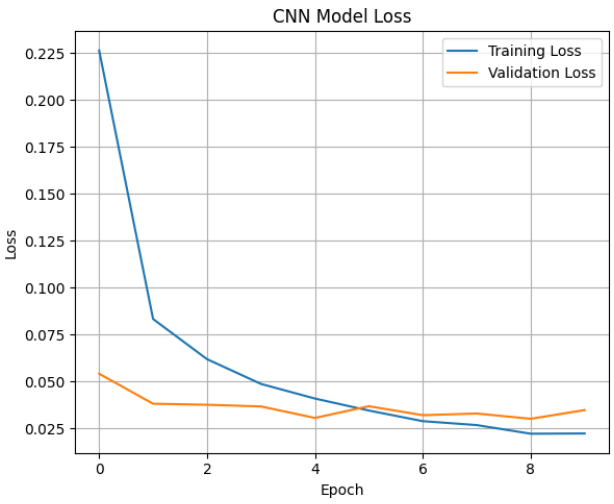
           0       0.99      1.00      0.99       980
           1       0.99      1.00      1.00      1135
           2       1.00      0.99      0.99      1032
           3       0.99      1.00      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.99      0.98      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000


--- CNN Confusion Matrix ---
```

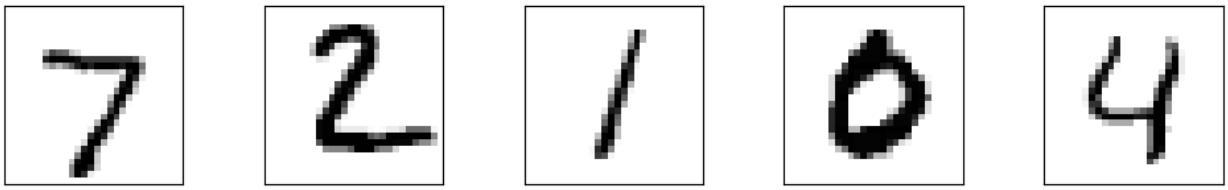







```
--- Sample CNN Predictions ---
```

Sample CNN Predictions (Green: Correct, Red: Incorrect)



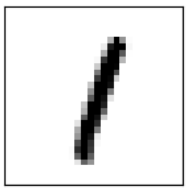
True: 7
Pred: 7

True: 2
Pred: 2

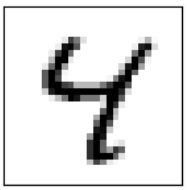
True: 1
Pred: 1

True: 0
Pred: 0

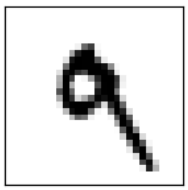
True: 4
Pred: 4



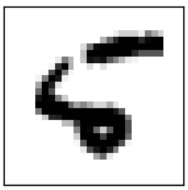
True: 1
Pred: 1



True: 4
Pred: 4



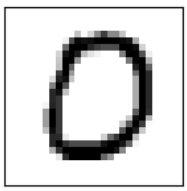
True: 9
Pred: 9



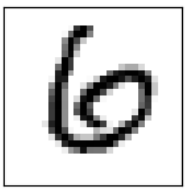
True: 5
Pred: 5



True: 9
Pred: 9



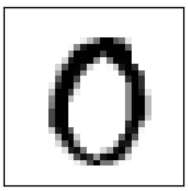
True: 0
Pred: 0



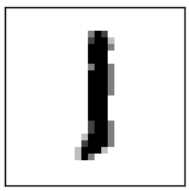
True: 6
Pred: 6



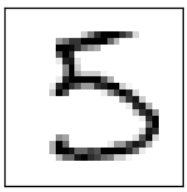
True: 9
Pred: 9



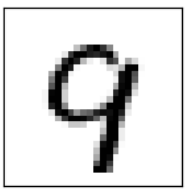
True: 0
Pred: 0



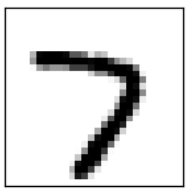
True: 1
Pred: 1



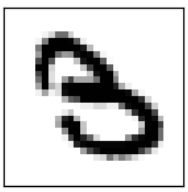
True: 5
Pred: 5



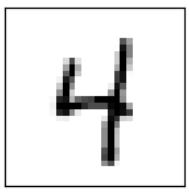
True: 9
Pred: 9



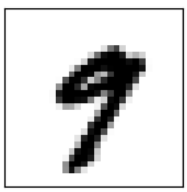
True: 7
Pred: 7



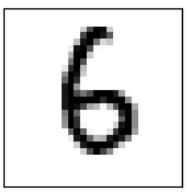
True: 3
Pred: 3



True: 4
Pred: 4



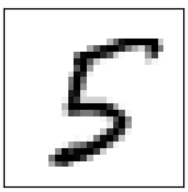
True: 9
Pred: 9



True: 6
Pred: 6



True: 6
Pred: 6



True: 5
Pred: 5



True: 4
Pred: 4